

Concepto 3

Spring Webflux.

Usando los proyectos que vimos en clase, con la carpeta de `..\springWebFlux\proyectos\...` siendo 01 webflux mono y 02 webflux flux, veremos lo que es la programación reactiva, donde `Mono<T>` es una computación asíncrona que produce 0 o 1 elemento, no más. Es similar a retornar un objeto individual, pero en vez de regresarlo inmediatamente, tenemos un pipeline reactivo que eventualmente produce el objeto.

En en nuestro archivo de `employeeRepository.java` adentro del proyecto webflux mono, vemos como con `Mono.justorEmpty` puede ser definido como el `Mono<T>` al contener un elemento o vacío, y simulamos la latencia del pipeline con `LATENCIA` de 5 segundos o 5000 millis. En nuestro archivo de `EmployeeRestController`, vemos como el GET, con el método find by ID ya no regresa un objeto empleado, sino un `Mono<Employee>` a Spring, donde este se suscribe al pipeline reactivo y manda la respuesta HTTP cuando el valor esta disponible. Con la latencia incluida, podemos ver que nuestra operación es asíncrona, teniendo esa tardanza de 5 segundos. También hay un GET by ID bloqueante, para demostrar que, con la misma consulta, el hilo se para esperando sin poder hacer nada, útil para ver la diferencia con los scripts provistos de pruebas `comparar.sh`, demostrando que con reactive los hilos no se bloquean, solo esperan hasta que la respuesta llegue, webflux mono puede tener múltiples peticiones progresando al mismo tiempo sin requerir un thread bloqueado por petición.

Para correr este proyecto, solamente se siguen las instrucciones para montarlo como los pasados, con comandos en powershell y similares montando la app en una ventana y pasando los requests en otra. Esta vez nuestro puerto esta en 8074, tenemos los `.sh` de prueba en la carpeta de scripts. (Cuando yo probe esto, me salieron valores bien largos, por alrededor de reactivo ~25 segundos, y bloqueante de ~160 segundos. Asumo que por que mi CPU esta sobreocupada con otras aplicaciones en mi equipo)

Pasando de `Mono` a `Flux<T>`, en nuestra carpeta `\02-webflux-flux` pasamos de leer un solo elemento a una secuencia de lecturas de sensores especificada en nuestro `SensorService.java`, con una cadencia de 1 segundo. `Flux` acepta de 0 a varios elementos, y lo usamos para representar un sensor con mediciones continuas, incluso infinitas, a través del paso del tiempo, cosa que no se representa fácilmente digamos en una lista.

En nuestro `LecturaRestController.java` vemos esto aplicado, simulando como con el primer endpoint de `comoJson`, Spring se suscribe, junta 5 lecturas y las suelta de un jalón a los 5 segundos, es decir que nuestra pantalla queda en blanco por 5 segundos. Pero si lo adaptamos como stream, podemos hacer una lectura por segundo como llegue el flux.

También hay operaciones que manejan el flujo sin detenerlo, como el que solo emite si pasa por encima de un umbral que en este caso es 30, o como con otro lee sin emitir hasta que pasa por debajo del umbral provisto, emite un `onComplete` y cierra el mismo la conexión.

Además, también vemos como un flux de lectura de datos puede convertirse a mono agregando múltiples elementos a un resultado individual, como se ve en `Mono<Resumen>`, que toma el `Flux<Lectura>` (en este caso 10 entradas), lo pasa a un `collectList` haciéndolo un elemento Mono y lo mapea con estadísticas, una función sacando datos de los valores.

Aquí, este proyecto también contiene los scripts a probar en `probar.sh` en su carpeta correspondiente. Y como siempre, este proyecto se monta en powershell con `.\mvnw.cmd spring-boot:run` en su carpeta correspondiente en una terminal, y pasar las peticiones en otra ubicada en la misma carpeta.

En resumen, la programación reactiva evita el problema que se puede presentar al tener varias operaciones simultaneas que se sobrepasan la capacidad de un CPU, evitando que se dejen hilos sin hacer nada, al programar con Mono y Flux que son capaces de representar resultados de manera asíncrona.